Programming Fundamentals

Original practice text for testing ingestion.

Chapter 1: Variables

1.1 Basics of variables

The function updates the value before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 71 items. The counter stores the loop before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 67 items. The buffer updates the value before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 58 items. The function copies the counter before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 14 items.

The value updates the record before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 31 items. The record moves the value before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 9 items. The buffer returns the value before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 40 items. The function updates the index before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 76 items. The pointer returns the string before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 16 items. The record copies the record before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 50 items.

The string stores the counter before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 82 items. The buffer moves the array before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 43 items. The array prints the record before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 49 items.

The buffer copies the index before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 13 items. The record prints the pointer before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 46 items. The array updates the pointer before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 18 items. The string returns the function before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 99 items. The loop prints the index before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 56 items.

- Use a clear name for every value in chapter 1.
- Use a clear name for every counter in chapter 1.
- Use a clear name for every string in chapter 1.

The record compares the loop before the next step, and example 1.9 shows why this order matters when the program runs on a small input of 91 items. The loop prints the record before the next step, and example 1.9 shows why this order matters when the program runs on a small input of 77 items.[1]

1.2 In practice of variables

The counter prints the pointer before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 92 items. The counter checks the value before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 85 items. The record checks the array before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 94 items.

The loop prints the value before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 48 items. The index updates the record before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 66 items. The value checks the buffer before the next step, and example 1.1 shows why this order matters when the program runs on a small

1 This rule also holds for the exercises at the end of chapter 1.

input of 19 items. The buffer moves the function before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 66 items. The counter prints the index before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 54 items. The string returns the pointer before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 58 items.

The function moves the loop before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 32 items. The index returns the counter before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 22 items. The buffer stores the buffer before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 65 items. The record checks the index before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 39 items. The value moves the index before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 71 items.

The record compares the record before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 19 items. The string stores the record before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 61 items. The string moves the function before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 54 items. The function prints the counter before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 84 items. The function copies the value before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 11 items.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > 0) {
            total += a[i];
        }
    }
    return total;
}
```

Listing 1.1: Adding the positive values of an array.

1. Read the input.
2. Check every value.
3. Print the total.

Operator	Meaning	Example
+	addition	a + b
%	remainder	8 % 3
==	equal to	a == b

1.3 Pitfalls of variables

The index compares the counter before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 79 items. The value stores the counter before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 75 items. The index updates the string before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 49 items. The record updates the value before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 29 items. The record returns the function before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 84 items. The pointer compares the loop before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 63 items.

The counter prints the array before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 64 items. The array updates the pointer before the next step, and example

1.1 shows why this order matters when the program runs on a small input of 21 items. The counter checks the loop before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 64 items.

The string copies the value before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 70 items. The loop stores the index before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 70 items. The pointer checks the counter before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 69 items. The loop compares the index before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 31 items.

Why variables go wrong

The buffer copies the record before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 33 items. The function copies the buffer before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 69 items. The array stores the loop before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 6 items. The pointer checks the array before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 27 items. The record prints the loop before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 95 items.

Mistake	Effect
off by one	skips a value
no base case	never stops

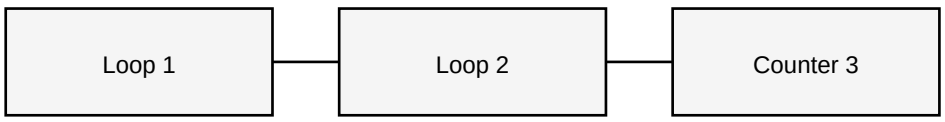

Figure 1.1: How the variables pass data along.

Figure 1.2: Running time for eight input sizes.

The array updates the loop before the next step, and example 1.20 shows why this order matters when the program runs on a small input of 87 items. The counter copies the function before the next step, and example 1.20 shows why this order matters when the program runs on a small input of 64 items. The index compares the function before the next step, and example 1.20 shows why this order matters when the program runs on a small input of 14 items.

The array updates the function before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 95 items. The index returns the index before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 6 items. The index prints the

record before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 86 items. The index prints the record before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 87 items. The loop returns the index before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 5 items. The value returns the counter before the next step, and example 1.21 shows why this order matters when the program runs on a small input of 58 items.

Glossary for chapter 1

Buffer 1: The buffer checks the value before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 30 items. The pointer copies the string before the next step, and example 1.0 shows why this order matters when the program runs on a small input of 78 items.

Loop 2: The pointer moves the string before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 19 items. The value prints the loop before the next step, and example 1.1 shows why this order matters when the program runs on a small input of 87 items.

Record 3: The string returns the function before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 71 items. The index stores the string before the next step, and example 1.2 shows why this order matters when the program runs on a small input of 59 items.

Index 4: The record returns the value before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 25 items. The index updates the array before the next step, and example 1.3 shows why this order matters when the program runs on a small input of 74 items.

Value 5: The loop prints the string before the next step, and example 1.4 shows why this order matters when the program runs on a small input of 16 items. The string copies the value before the next step, and example 1.4 shows why this order matters when the program runs on a small input of 27 items.

Pointer 6: The value prints the counter before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 74 items. The value prints the counter before the next step, and example 1.5 shows why this order matters when the program runs on a small input of 44 items.

Record 7: The string copies the record before the next step, and example 1.6 shows why this order matters when the program runs on a small input of 91 items. The pointer prints the array before the next step, and example 1.6 shows why this order matters when the program runs on a small input of 67 items.

Buffer 8: The string copies the pointer before the next step, and example 1.7 shows why this order matters when the program runs on a small input of 60 items. The index updates the function before the next step, and example 1.7 shows why this order matters when the program runs on a small input of 53 items.

Array 9: The loop copies the counter before the next step, and example 1.8 shows why this order matters when the program runs on a small input of 57 items. The counter checks the buffer before the next step, and example 1.8 shows why this order matters when the program runs on a small input of 18 items.

Index 10: The loop checks the index before the next step, and example 1.9 shows why this order matters when the program runs on a small input of 20 items. The array updates the buffer before the next step, and example 1.9 shows why this order matters when the program runs on a small input of 53 items.

Array 11: The index returns the buffer before the next step, and example 1.10 shows why this order matters when the program runs on a small input of 93 items. The function moves the string before the next step, and example 1.10 shows why this order matters when the program runs on a small input of 46 items.

Function 12: The buffer compares the loop before the next step, and example 1.11 shows why this order matters when the program runs on a small input of 14 items. The loop compares the value before the next step, and example 1.11 shows why this order matters when the program runs on a small input of 73 items.

Array 13: The array moves the value before the next step, and example 1.12 shows why this order matters when the program runs on a small input of 45 items. The string checks the record before the next step, and example 1.12 shows why this order matters when the program runs on a small input of 68 items.

Counter 14: The counter updates the buffer before the next step, and example 1.13 shows why this order matters when the program runs on a small input of 13 items. The pointer stores the pointer before the next step, and example 1.13 shows why this order matters when the program runs on a small input of 26 items.

Chapter 2: Loops

2.1 Basics of loops

The function moves the pointer before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 22 items. The string prints the string before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 92 items. The loop checks the counter before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 10 items. The index updates the function before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 37 items.

The counter updates the pointer before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 80 items. The buffer checks the counter before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 18 items. The array compares the value before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 73 items.

The pointer returns the record before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 8 items. The string updates the buffer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 23 items. The pointer returns the value before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 28 items. The pointer copies the pointer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 40 items. The array returns the string before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 37 items. The loop checks the value before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 7 items.

The value copies the string before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 68 items. The array prints the buffer before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 16 items. The function moves the array before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 67 items.

- Use a clear name for every pointer in chapter 2.
- Use a clear name for every buffer in chapter 2.
- Use a clear name for every buffer in chapter 2.

The loop returns the buffer before the next step, and example 2.9 shows why this order matters when the program runs on a small input of 54 items. The loop returns the value before the next step, and example 2.9 shows why this order matters when the program runs on a small input of 4 items.[1]

2.2 In practice of loops

The function stores the index before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 13 items. The function checks the string before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 79 items. The buffer stores the pointer before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 61 items. The index checks the index before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 60 items. The value compares the pointer before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 45 items.

The buffer checks the value before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 30 items. The loop stores the index before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 45 items. The function prints the counter before the next step, and example 2.1 shows why this order matters when the program runs on a

1 This rule also holds for the exercises at the end of chapter 2.

small input of 38 items. The string copies the buffer before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 67 items. The value checks the counter before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 14 items.

The function stores the record before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 53 items. The value checks the pointer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 83 items. The buffer returns the counter before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 87 items. The record compares the function before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 95 items.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > 0) {
            total += a[i];
        }
    }
    return total;
}
```

Listing 2.1: Adding the positive values of an array.

1. Read the input.
2. Check every value.
3. Print the total.

Operator	Meaning	Example
+	addition	a + b
%	remainder	9 % 3
==	equal to	a == b

2.3 Pitfalls of loops

The pointer returns the record before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 8 items. The string returns the function before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 70 items. The string stores the record before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 90 items. The record updates the buffer before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 6 items.

The index updates the loop before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 51 items. The array stores the string before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 83 items. The value copies the string before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 65 items.

The value updates the array before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 98 items. The string updates the string before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 87 items. The string prints the counter before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 35 items. The counter copies the pointer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 96 items. The buffer prints the buffer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 66 items.

The counter checks the array before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 8 items. The record updates the buffer before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 79 items. The index checks the

loop before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 86 items. The pointer returns the record before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 4 items. The array prints the value before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 37 items. The counter prints the buffer before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 40 items.

Why loops go wrong

The array prints the array before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 18 items. The string checks the buffer before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 13 items. The array checks the value before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 61 items. The counter prints the string before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 37 items. The function copies the buffer before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 12 items.

Mistake	Effect
off by one	skips a value
no base case	never stops

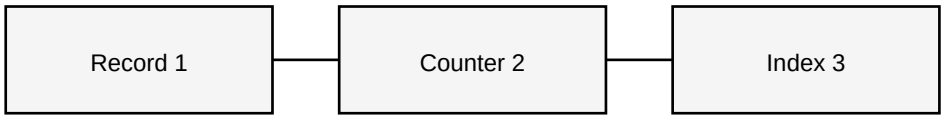

Figure 2.1: How the loops pass data along.

Figure 2.2: Running time for eight input sizes.

The counter copies the loop before the next step, and example 2.20 shows why this order matters when the program runs on a small input of 66 items. The array stores the function before the next step, and example 2.20 shows why this order matters when the program runs on a small input of 23 items. The value prints the array before the next step, and example 2.20 shows why this order matters when the program runs on a small input of 54 items. The pointer moves the index before the next step, and example 2.20 shows why this order matters when the program runs on a small input of 47 items. The function updates the loop before the next step, and example 2.20 shows why this order matters when the program runs on a small input of 45 items.

The loop moves the loop before the next step, and example 2.21 shows why this order matters when the program runs on a small input of 18 items. The buffer checks the value before the next step, and example 2.21 shows why this order matters when the program runs on a small input of 35 items. The loop moves the

counter before the next step, and example 2.21 shows why this order matters when the program runs on a small input of 52 items.

Glossary for chapter 2

Record 1: The counter moves the loop before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 99 items. The pointer checks the value before the next step, and example 2.0 shows why this order matters when the program runs on a small input of 16 items.

Value 2: The pointer copies the index before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 37 items. The function compares the string before the next step, and example 2.1 shows why this order matters when the program runs on a small input of 27 items.

Loop 3: The function moves the value before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 73 items. The string updates the buffer before the next step, and example 2.2 shows why this order matters when the program runs on a small input of 9 items.

Function 4: The array returns the record before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 85 items. The pointer stores the array before the next step, and example 2.3 shows why this order matters when the program runs on a small input of 73 items.

Index 5: The index moves the array before the next step, and example 2.4 shows why this order matters when the program runs on a small input of 46 items. The pointer checks the pointer before the next step, and example 2.4 shows why this order matters when the program runs on a small input of 97 items.

Pointer 6: The function checks the buffer before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 64 items. The string updates the function before the next step, and example 2.5 shows why this order matters when the program runs on a small input of 24 items.

Index 7: The counter prints the buffer before the next step, and example 2.6 shows why this order matters when the program runs on a small input of 73 items. The buffer compares the array before the next step, and example 2.6 shows why this order matters when the program runs on a small input of 60 items.

Function 8: The index copies the string before the next step, and example 2.7 shows why this order matters when the program runs on a small input of 34 items. The counter compares the index before the next step, and example 2.7 shows why this order matters when the program runs on a small input of 74 items.

Counter 9: The loop compares the buffer before the next step, and example 2.8 shows why this order matters when the program runs on a small input of 36 items. The record stores the buffer before the next step, and example 2.8 shows why this order matters when the program runs on a small input of 98 items.

Function 10: The function copies the function before the next step, and example 2.9 shows why this order matters when the program runs on a small input of 51 items. The pointer stores the loop before the next step, and example 2.9 shows why this order matters when the program runs on a small input of 66 items.

Pointer 11: The record returns the loop before the next step, and example 2.10 shows why this order matters when the program runs on a small input of 90 items. The string copies the string before the next step, and example 2.10 shows why this order matters when the program runs on a small input of 14 items.

Pointer 12: The buffer moves the function before the next step, and example 2.11 shows why this order matters when the program runs on a small input of 85 items. The array checks the function before the next step, and example 2.11 shows why this order matters when the program runs on a small input of 5 items.

Index 13: The value prints the function before the next step, and example 2.12 shows why this order matters when the program runs on a small input of 78 items. The array updates the value before the next step, and example 2.12 shows why this order matters when the program runs on a small input of 53 items.

String 14: The array copies the array before the next step, and example 2.13 shows why this order matters when the program runs on a small input of 16 items. The buffer returns the index before the next step, and example 2.13 shows why this order matters when the program runs on a small input of 69 items.

Chapter 3: Functions

3.1 Basics of functions

The array stores the counter before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 3 items. The index stores the buffer before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 85 items. The pointer checks the index before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 70 items.

The counter updates the counter before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 41 items. The string copies the record before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 52 items. The pointer stores the buffer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 4 items. The string prints the pointer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 38 items. The loop prints the buffer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 70 items. The buffer copies the string before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 6 items.

The pointer stores the value before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 27 items. The array updates the function before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 35 items. The buffer compares the function before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 32 items. The array compares the value before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 94 items. The function moves the loop before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 28 items. The value updates the pointer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 29 items.

The buffer copies the pointer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 32 items. The array checks the buffer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 40 items. The counter prints the record before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 81 items. The index prints the buffer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 56 items. The value returns the record before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 53 items. The value stores the buffer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 79 items.

The function stores the value before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 26 items. The function compares the array before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 96 items. The counter returns the counter before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 45 items. The buffer prints the index before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 7 items.

- Use a clear name for every pointer in chapter 3.
- Use a clear name for every function in chapter 3.
- Use a clear name for every loop in chapter 3.

The loop returns the array before the next step, and example 3.9 shows why this order matters when the program runs on a small input of 16 items. The value checks the counter before the next step, and example 3.9 shows why this order matters when the program runs on a small input of 13 items.[1]

1 This rule also holds for the exercises at the end of chapter 3.

3.2 In practice of functions

The counter copies the string before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 51 items. The loop moves the pointer before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 14 items. The value copies the array before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 50 items. The string copies the array before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 44 items. The loop stores the array before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 83 items. The function moves the buffer before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 8 items.

The value updates the array before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 10 items. The pointer updates the buffer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 80 items. The loop checks the loop before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 45 items. The record checks the value before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 98 items. The loop checks the pointer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 3 items. The record stores the counter before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 32 items.

The array moves the array before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 35 items. The function returns the array before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 66 items. The index checks the value before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 91 items.

The record compares the buffer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 43 items. The array updates the loop before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 68 items. The buffer returns the function before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 34 items. The function stores the counter before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 64 items.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > 0) {
            total += a[i];
        }
    }
    return total;
}
```

Listing 3.1: Adding the positive values of an array.

1. Read the input.
2. Check every value.
3. Print the total.

Operator	Meaning	Example
+	addition	a + b
%	remainder	10 % 3
==	equal to	a == b

3.3 Pitfalls of functions

The index updates the function before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 12 items. The pointer updates the record before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 29 items. The counter prints the function before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 93 items. The array copies the index before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 20 items. The function copies the array before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 98 items.

The pointer checks the pointer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 75 items. The pointer checks the loop before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 97 items. The pointer prints the buffer before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 34 items.

The buffer returns the buffer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 39 items. The record compares the buffer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 11 items. The function copies the pointer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 67 items. The string updates the buffer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 86 items.

The value stores the counter before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 63 items. The buffer compares the array before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 8 items. The pointer updates the buffer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 9 items. The buffer copies the record before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 12 items. The loop returns the string before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 60 items. The record stores the pointer before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 16 items.

The buffer compares the value before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 46 items. The index copies the value before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 35 items. The value copies the record before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 4 items. The loop compares the function before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 26 items. The record updates the pointer before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 29 items.

Why functions go wrong

The array prints the string before the next step, and example 3.5 shows why this order matters when the program runs on a small input of 11 items. The function moves the counter before the next step, and example 3.5 shows why this order matters when the program runs on a small input of 87 items. The string updates the index before the next step, and example 3.5 shows why this order matters when the program runs on a small input of 86 items.

Mistake	Effect
off by one	skips a value
no base case	never stops

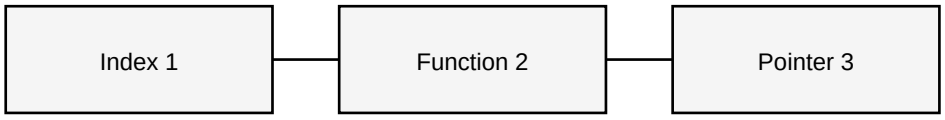

Figure 3.1: How the functions pass data along.

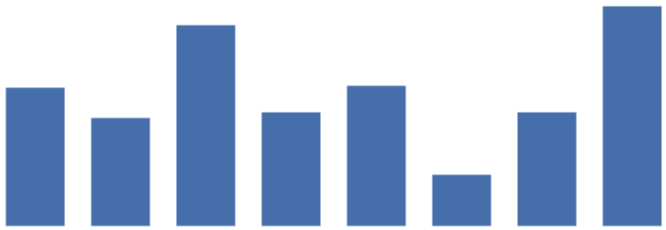

Figure 3.2: Running time for eight input sizes.

The function stores the function before the next step, and example 3.20 shows why this order matters when the program runs on a small input of 49 items. The buffer moves the function before the next step, and example 3.20 shows why this order matters when the program runs on a small input of 29 items. The value returns the function before the next step, and example 3.20 shows why this order matters when the program runs on a small input of 57 items. The counter moves the counter before the next step, and example 3.20 shows why this order matters when the program runs on a small input of 76 items. The loop returns the array before the next step, and example 3.20 shows why this order matters when the program runs on a small input of 19 items.

The value returns the string before the next step, and example 3.21 shows why this order matters when the program runs on a small input of 85 items. The function compares the counter before the next step, and example 3.21 shows why this order matters when the program runs on a small input of 97 items. The string returns the index before the next step, and example 3.21 shows why this order matters when the program runs on a small input of 47 items.

Glossary for chapter 3

Pointer 1: The index returns the string before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 11 items. The counter prints the function before the next step, and example 3.0 shows why this order matters when the program runs on a small input of 99 items.

Buffer 2: The pointer stores the index before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 64 items. The loop moves the value before the next step, and example 3.1 shows why this order matters when the program runs on a small input of 14 items.

Record 3: The index moves the buffer before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 81 items. The buffer returns the array before the next step, and example 3.2 shows why this order matters when the program runs on a small input of 75 items.

Buffer 4: The value returns the function before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 52 items. The loop returns the counter before the next step, and example 3.3 shows why this order matters when the program runs on a small input of 34 items.

Buffer 5: The value stores the string before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 88 items. The loop moves the counter before the next step, and example 3.4 shows why this order matters when the program runs on a small input of 79 items.

Array 6: The string moves the pointer before the next step, and example 3.5 shows why this order matters when the program runs on a small input of 42 items. The record moves the buffer before the next step, and example 3.5 shows why this order matters when the program runs on a small input of 52 items.

Loop 7: The array prints the string before the next step, and example 3.6 shows why this order matters when the program runs on a small input of 25 items. The value prints the value before the next step, and example 3.6 shows why this order matters when the program runs on a small input of 62 items.

Buffer 8: The array prints the record before the next step, and example 3.7 shows why this order matters when the program runs on a small input of 25 items. The array updates the function before the next step, and example 3.7 shows why this order matters when the program runs on a small input of 11 items.

Index 9: The loop compares the function before the next step, and example 3.8 shows why this order matters when the program runs on a small input of 14 items. The array stores the string before the next step, and example 3.8 shows why this order matters when the program runs on a small input of 8 items.

Index 10: The counter updates the loop before the next step, and example 3.9 shows why this order matters when the program runs on a small input of 9 items. The string returns the function before the next step, and example 3.9 shows why this order matters when the program runs on a small input of 6 items.

Counter 11: The record copies the counter before the next step, and example 3.10 shows why this order matters when the program runs on a small input of 19 items. The array returns the pointer before the next step, and example 3.10 shows why this order matters when the program runs on a small input of 90 items.

Buffer 12: The counter checks the loop before the next step, and example 3.11 shows why this order matters when the program runs on a small input of 23 items. The loop checks the record before the next step, and example 3.11 shows why this order matters when the program runs on a small input of 61 items.

Index 13: The pointer prints the string before the next step, and example 3.12 shows why this order matters when the program runs on a small input of 29 items. The record copies the pointer before the next step, and example 3.12 shows why this order matters when the program runs on a small input of 43 items.

Loop 14: The value returns the buffer before the next step, and example 3.13 shows why this order matters when the program runs on a small input of 54 items. The index compares the pointer before the next step, and example 3.13 shows why this order matters when the program runs on a small input of 51 items.

Chapter 4: Arrays

4.1 Basics of arrays

The counter stores the string before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 84 items. The loop updates the array before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 35 items. The string compares the function before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 36 items. The function returns the loop before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 49 items. The loop prints the counter before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 32 items.

The record checks the value before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 69 items. The pointer compares the pointer before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 96 items. The value copies the value before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 22 items. The pointer moves the record before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 56 items.

The value prints the index before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 32 items. The record stores the value before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 9 items. The value compares the record before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 41 items. The counter compares the string before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 71 items. The buffer checks the function before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 78 items.

- Use a clear name for every index in chapter 4.
- Use a clear name for every buffer in chapter 4.
- Use a clear name for every loop in chapter 4.

The record returns the array before the next step, and example 4.9 shows why this order matters when the program runs on a small input of 20 items. The value returns the buffer before the next step, and example 4.9 shows why this order matters when the program runs on a small input of 60 items.[1]

4.2 In practice of arrays

The index moves the pointer before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 36 items. The value compares the value before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 79 items. The record prints the array before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 34 items.

The value stores the value before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 71 items. The value returns the function before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 33 items. The index updates the value before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 4 items. The record copies the string before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 21 items.

The buffer moves the string before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 81 items. The index checks the string before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 11 items. The pointer prints the value before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 94 items. The string moves the value before the next step, and example 4.2 shows why this order

1 This rule also holds for the exercises at the end of chapter 4.

matters when the program runs on a small input of 58 items. The array prints the counter before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 25 items. The buffer checks the counter before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 32 items.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > 0) {
            total += a[i];
        }
    }
    return total;
}
```

Listing 4.1: Adding the positive values of an array.

1. Read the input.
2. Check every value.
3. Print the total.

Operator	Meaning	Example
+	addition	a + b
%	remainder	11 % 3
==	equal to	a == b

4.3 Pitfalls of arrays

The counter checks the loop before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 94 items. The value moves the pointer before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 90 items. The string checks the pointer before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 85 items.

The counter stores the string before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 24 items. The pointer copies the buffer before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 23 items. The loop moves the buffer before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 45 items. The record moves the buffer before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 83 items.

The array stores the string before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 6 items. The function checks the buffer before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 30 items. The function updates the record before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 75 items. The index stores the index before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 6 items. The counter returns the counter before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 47 items. The index stores the value before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 8 items.

The value stores the counter before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 11 items. The record copies the loop before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 71 items. The counter updates the function before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 34 items. The buffer updates the buffer before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 7 items.

The counter prints the pointer before the next step, and example 4.4 shows why this order matters when the program runs on a small input of 15 items. The index copies the counter before the next step, and example 4.4 shows why this order matters when the program runs on a small input of 40 items. The loop moves the loop before the next step, and example 4.4 shows why this order matters when the program runs on a small input of 36 items.

Why arrays go wrong

The loop checks the pointer before the next step, and example 4.5 shows why this order matters when the program runs on a small input of 9 items. The loop prints the loop before the next step, and example 4.5 shows why this order matters when the program runs on a small input of 39 items. The record moves the value before the next step, and example 4.5 shows why this order matters when the program runs on a small input of 6 items.

Mistake	Effect
off by one	skips a value
no base case	never stops

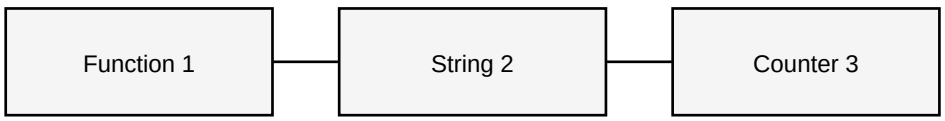

Figure 4.1: How the arrays pass data along.

Figure 4.2: Running time for eight input sizes.

The record returns the pointer before the next step, and example 4.20 shows why this order matters when the program runs on a small input of 58 items. The value copies the string before the next step, and example 4.20 shows why this order matters when the program runs on a small input of 39 items. The value compares the value before the next step, and example 4.20 shows why this order matters when the program runs on a small input of 65 items.

The array prints the index before the next step, and example 4.21 shows why this order matters when the program runs on a small input of 78 items. The loop checks the string before the next step, and example 4.21 shows why this order matters when the program runs on a small input of 76 items. The index copies the pointer before the next step, and example 4.21 shows why this order matters when the program runs on a small input of 92 items.

Glossary for chapter 4

Buffer 1: The array updates the index before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 84 items. The counter updates the array before the next step, and example 4.0 shows why this order matters when the program runs on a small input of 83 items.

Loop 2: The loop moves the counter before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 53 items. The counter stores the function before the next step, and example 4.1 shows why this order matters when the program runs on a small input of 50 items.

Buffer 3: The pointer moves the pointer before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 72 items. The string moves the index before the next step, and example 4.2 shows why this order matters when the program runs on a small input of 83 items.

Buffer 4: The array stores the index before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 47 items. The record returns the loop before the next step, and example 4.3 shows why this order matters when the program runs on a small input of 60 items.

String 5: The loop prints the index before the next step, and example 4.4 shows why this order matters when the program runs on a small input of 59 items. The pointer copies the record before the next step, and example 4.4 shows why this order matters when the program runs on a small input of 19 items.

Loop 6: The array copies the buffer before the next step, and example 4.5 shows why this order matters when the program runs on a small input of 37 items. The pointer returns the record before the next step, and example 4.5 shows why this order matters when the program runs on a small input of 95 items.

Index 7: The buffer compares the loop before the next step, and example 4.6 shows why this order matters when the program runs on a small input of 23 items. The buffer copies the loop before the next step, and example 4.6 shows why this order matters when the program runs on a small input of 36 items.

Counter 8: The index copies the counter before the next step, and example 4.7 shows why this order matters when the program runs on a small input of 52 items. The index checks the index before the next step, and example 4.7 shows why this order matters when the program runs on a small input of 96 items.

Pointer 9: The function copies the pointer before the next step, and example 4.8 shows why this order matters when the program runs on a small input of 16 items. The counter copies the pointer before the next step, and example 4.8 shows why this order matters when the program runs on a small input of 52 items.

Array 10: The value moves the value before the next step, and example 4.9 shows why this order matters when the program runs on a small input of 58 items. The buffer checks the string before the next step, and example 4.9 shows why this order matters when the program runs on a small input of 62 items.

Value 11: The index moves the pointer before the next step, and example 4.10 shows why this order matters when the program runs on a small input of 3 items. The buffer moves the function before the next step, and example 4.10 shows why this order matters when the program runs on a small input of 32 items.

Record 12: The buffer updates the index before the next step, and example 4.11 shows why this order matters when the program runs on a small input of 61 items. The function checks the loop before the next step, and example 4.11 shows why this order matters when the program runs on a small input of 83 items.

Counter 13: The function moves the buffer before the next step, and example 4.12 shows why this order matters when the program runs on a small input of 94 items. The index moves the pointer before the next step, and example 4.12 shows why this order matters when the program runs on a small input of 64 items.

Array 14: The value moves the record before the next step, and example 4.13 shows why this order matters when the program runs on a small input of 69 items. The index stores the loop before the next step, and example 4.13 shows why this order matters when the program runs on a small input of 52 items.

Chapter 5: Pointers

5.1 Basics of pointers

The value copies the pointer before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 23 items. The buffer compares the string before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 15 items. The record copies the array before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 94 items.

The string compares the value before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 69 items. The loop prints the function before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 29 items. The index updates the function before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 96 items. The record stores the loop before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 35 items. The pointer moves the function before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 10 items. The value moves the counter before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 56 items.

The record updates the pointer before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 31 items. The pointer copies the function before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 53 items. The array returns the buffer before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 19 items. The counter prints the buffer before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 85 items. The string returns the buffer before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 48 items.

The array returns the pointer before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 63 items. The loop checks the buffer before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 93 items. The function moves the pointer before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 89 items. The index stores the array before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 95 items. The pointer copies the loop before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 86 items. The pointer prints the loop before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 65 items.

- Use a clear name for every function in chapter 5.
- Use a clear name for every record in chapter 5.
- Use a clear name for every counter in chapter 5.

The loop checks the index before the next step, and example 5.9 shows why this order matters when the program runs on a small input of 52 items. The value compares the counter before the next step, and example 5.9 shows why this order matters when the program runs on a small input of 20 items.[1]

5.2 In practice of pointers

The record stores the value before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 29 items. The counter checks the pointer before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 80 items. The counter returns the record before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 32 items. The index compares the array before the next step, and example 5.0 shows why this

1 This rule also holds for the exercises at the end of chapter 5.

order matters when the program runs on a small input of 22 items. The buffer returns the function before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 81 items.

The string copies the pointer before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 66 items. The buffer updates the string before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 97 items. The array updates the counter before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 36 items.

The buffer prints the index before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 66 items. The string prints the value before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 62 items. The index copies the array before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 66 items. The index stores the string before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 23 items. The loop prints the array before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 88 items. The pointer compares the array before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 57 items.

The counter compares the index before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 84 items. The value stores the value before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 90 items. The loop prints the counter before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 65 items. The index copies the value before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 94 items. The function compares the index before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 15 items. The loop prints the loop before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 70 items.

The pointer compares the function before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 57 items. The pointer stores the string before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 40 items. The pointer prints the loop before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 54 items. The loop checks the string before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 67 items.

```
int sum(const int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > 0) {
            total += a[i];
        }
    }
    return total;
}
```

Listing 5.1: Adding the positive values of an array.

1. Read the input.
2. Check every value.
3. Print the total.

Operator	Meaning	Example
+	addition	a + b
%	remainder	12 % 3
==	equal to	a == b

5.3 Pitfalls of pointers

The array compares the counter before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 27 items. The loop returns the pointer before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 78 items. The counter moves the value before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 95 items. The string stores the function before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 54 items.

The counter stores the value before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 27 items. The array stores the record before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 67 items. The string moves the record before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 81 items. The index updates the record before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 30 items. The value returns the array before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 15 items.

The value updates the function before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 86 items. The value returns the loop before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 42 items. The string checks the pointer before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 26 items. The function compares the value before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 5 items.

The record stores the record before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 66 items. The record stores the string before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 18 items. The function moves the record before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 60 items. The counter moves the value before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 79 items. The record prints the index before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 55 items. The string updates the counter before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 85 items.

Why pointers go wrong

The buffer stores the index before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 57 items. The value updates the value before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 14 items. The buffer returns the counter before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 63 items. The value copies the pointer before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 60 items. The index compares the value before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 98 items. The index checks the counter before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 83 items.

Mistake	Effect
off by one	skips a value
no base case	never stops

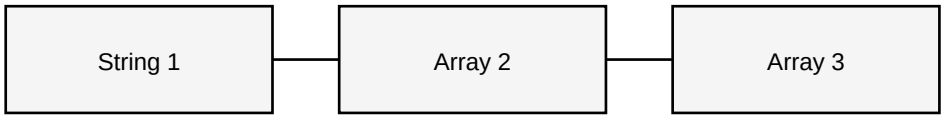

Figure 5.1: How the pointers pass data along.

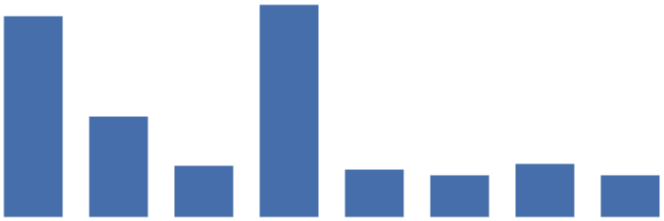

Figure 5.2: Running time for eight input sizes.

The function checks the pointer before the next step, and example 5.20 shows why this order matters when the program runs on a small input of 96 items. The record prints the index before the next step, and example 5.20 shows why this order matters when the program runs on a small input of 80 items. The value compares the loop before the next step, and example 5.20 shows why this order matters when the program runs on a small input of 76 items.

The array returns the index before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 17 items. The loop moves the index before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 64 items. The function checks the array before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 99 items. The record checks the loop before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 38 items. The value compares the record before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 80 items. The value checks the index before the next step, and example 5.21 shows why this order matters when the program runs on a small input of 77 items.

Glossary for chapter 5

Function 1: The buffer moves the function before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 90 items. The function copies the record before the next step, and example 5.0 shows why this order matters when the program runs on a small input of 60 items.

Pointer 2: The value checks the loop before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 37 items. The function stores the index before the next step, and example 5.1 shows why this order matters when the program runs on a small input of 39 items.

Index 3: The record checks the index before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 73 items. The array updates the loop before the next step, and example 5.2 shows why this order matters when the program runs on a small input of 72 items.

String 4: The array copies the function before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 99 items. The buffer stores the pointer before the next step, and example 5.3 shows why this order matters when the program runs on a small input of 89 items.

Function 5: The array checks the buffer before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 78 items. The value prints the function before the next step, and example 5.4 shows why this order matters when the program runs on a small input of 72 items.

Counter 6: The string updates the loop before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 32 items. The function checks the record before the next step, and example 5.5 shows why this order matters when the program runs on a small input of 69 items.

Loop 7: The array copies the string before the next step, and example 5.6 shows why this order matters when the program runs on a small input of 27 items. The buffer updates the buffer before the next step, and example 5.6 shows why this order matters when the program runs on a small input of 26 items.

Pointer 8: The loop compares the record before the next step, and example 5.7 shows why this order matters when the program runs on a small input of 54 items. The string copies the index before the next step, and example 5.7 shows why this order matters when the program runs on a small input of 8 items.

Array 9: The loop compares the counter before the next step, and example 5.8 shows why this order matters when the program runs on a small input of 83 items. The array returns the counter before the next step, and example 5.8 shows why this order matters when the program runs on a small input of 43 items.

Record 10: The value checks the loop before the next step, and example 5.9 shows why this order matters when the program runs on a small input of 69 items. The record updates the value before the next step, and example 5.9 shows why this order matters when the program runs on a small input of 7 items.

Buffer 11: The record copies the array before the next step, and example 5.10 shows why this order matters when the program runs on a small input of 36 items. The pointer updates the function before the next step, and example 5.10 shows why this order matters when the program runs on a small input of 60 items.

Record 12: The record checks the index before the next step, and example 5.11 shows why this order matters when the program runs on a small input of 7 items. The loop returns the buffer before the next step, and example 5.11 shows why this order matters when the program runs on a small input of 51 items.

Counter 13: The value stores the value before the next step, and example 5.12 shows why this order matters when the program runs on a small input of 74 items. The loop prints the array before the next step, and example 5.12 shows why this order matters when the program runs on a small input of 11 items.

Record 14: The function updates the counter before the next step, and example 5.13 shows why this order matters when the program runs on a small input of 35 items. The loop copies the record before the next step, and example 5.13 shows why this order matters when the program runs on a small input of 85 items.